Nifty ORB Alert System

Operator Handbook - V3

Paper trading | Auto SL/Target/Timeout | Trade journal

May 2026

Nifty ORB Alert System — Handbook (V3)

Your daily operating manual for the breakout alert + 1-tap order bot running on AWS Lightsail.

What this system does

Every trading day, this bot watches the Nifty current-month futures contract on Zerodha and tells you (via Telegram) when price makes a clean breakout from the first 15-minute range of the day. When a signal fires, the alert comes with a tappable BUY button that places an ITM-1 option order at a marketable LIMIT price — one tap, one lot, done.

-> ORB window: 9:15–9:30 IST. The high and low of that 15-min candle become your levels.

-> Watch period: 9:30 AM – 3:30 PM IST. Every 5 minutes (V2 — was 3 min in V1), the latest candle close is checked.

-> Trigger: A 5-min close above ORB high → buy ITM-1 CE (one strike below spot, ~0.65 delta). A close below ORB low → buy ITM-1 PE (one strike above spot).

-> Filters (to cut noise):

-> Volume on the breakout candle must be > 1.2× the average of the last 20 five-minute candles.

-> Close must be on the right side of intraday VWAP (above for UP, below for DOWN).

-> Re-fires: Up to 2 alerts per direction per day. After fire #1, price must pull back into the ORB by ≥ 15 pts before the system re-arms.

-> One-tap order: Each breakout alert includes an inline button: BUY 65 CE @ Rs.152.95. Tap it → bot places a LIMIT order via Kite. You still manage stop-loss and exits manually.

-> Safety guards: auto-order is skipped (with a "place manually" note) when option spread > 5%, premium > Rs.500, or quote depth is missing.

Pre-flight: the DRY_RUN flag

The bot ships with DRY_RUN = True in config.py. In this mode:

-> All alerts and buttons appear normally on Telegram

-> When you tap the button, the bot logs the order it would have placed and confirms back: [DRY_RUN] Order placed: NIFTY26MAY24450CE BUY 65 @ LIMIT Rs.152.95

-> No real money moves

Run for one full trading day in DRY_RUN. Verify each alert: is the symbol right (correct expiry, correct CE/PE), is the strike one ITM (50 pts in the money for Nifty), is the price near the visible Kite quote? Once you've seen 2-3 clean alerts pass that check, flip the flag:

```
ssh ubuntu@<lightsail-ip>
cd ~/nifty-orb-alerts
nano config.py        # change DRY_RUN = True  ->  DRY_RUN = False
pkill -f orb_monitor.py
nohup python orb_monitor.py > orb.log 2>&1 &
```

The startup Telegram message will now say ORB monitor started (LIVE) instead of (DRY_RUN).

Paper trading mode (V3)

Before flipping DRY_RUN = False (which uses real money), run PAPER_TRADE = True for a couple of weeks. Paper mode is the realistic dress rehearsal:

-> All alerts fire normally on Telegram with BUY buttons.

-> When you tap the button, no Kite order is placed — but the bot:

1. Journals the trade in trades.db at the alert-time LIMIT price (same value LIVE will use). 2. Starts monitoring the position every minute (POSITION_POLL_SECONDS = 60). 3. Auto-exits on SL (premium drops 30%), TARGET (premium rises 50%), or TIMEOUT (15:15 IST square-off). 4. Sends a Telegram exit alert with realized P&L.

This builds a full P&L history without risking capital. Mode truth table:

`PAPER_TRADE`	`DRY_RUN`	Mode	Real order?	DB row?	Auto-exit?
True	True	PAPER	No	Yes	Yes
True	False	PAPER	No	Yes	Yes
False	True	DRY_RUN	No	No	No (alert-only)
False	False	LIVE	Yes	Yes	Yes

Important: PAPER_TRADE overrides DRY_RUN. To go live, set both to False.

How the journal records prices

The journal records the price from the alert (e.g., LIMIT: Rs.152.95 shown on the BUY button), not the live ask at the moment you tap. This means:

-> In LIVE, your Kite LIMIT order is placed at exactly that price — fills land at or near it.

-> In PAPER, the same price is recorded — so the journal measures signal quality, not tap-reaction speed.

If you tap a button 5 minutes late and the option has moved Rs.30, the journal still uses the original alert price. The position monitor uses real-time LTP for SL / target / timeout decisions, so the trade is still evaluated against current market — but the entry P&L is anchored to the price the bot recommended.

Configurable exit levels (in config.py)

Setting	Default	Meaning
SL_PCT	0.30	Exit when premium drops 30% below entry
TARGET_PCT	0.50	Exit when premium rises 50% above entry
SQUARE_OFF_HOUR	15	Hard square-off hour (IST)
SQUARE_OFF_MINUTE	15	Hard square-off minute. 15:15 = 15 min before close
POSITION_POLL_SECONDS	60	How often to poll the option for exit conditions
RISK_PER_TRADE_INR	5000	Max rupees risked per trade; pricier options get skipped (see below)
MAX_LOTS	1	Hard ceiling on lots per trade — never sizes beyond one lot

MONTHLY_LOSS_LIMIT_INR	15000	Month's realized loss cap; pauses new trades when hit

Position sizing & the monthly loss limit (V4)

Two guards now bound how much you can lose. Both are automatic — you don't tap anything extra.

1. Per-trade risk cap (RISK_PER_TRADE_INR, default Rs.5,000). Before showing a BUY button, the bot works out the worst case if the stop-loss hits:

> one-lot risk = option premium × SL_PCT (0.30) × LOT_SIZE (65) = premium × 19.5

- -> If that worst-case loss is within Rs.5,000, you get the button for one lot (65).
- -> If even one lot would risk more than Rs.5,000, the trade is skipped — no button — and you'll see 1-lot risk Rs.… > risk cap Rs.5000.
- -> At Rs.5,000 the cutoff is a premium of about Rs.256: cheaper options trade, pricier ones are skipped.

Why one lot only? MAX_LOTS = 1 is a hard ceiling so the cap can only ever *skip* a trade, never *scale you up* into a bigger position. This is deliberate downside protection. If you ever want the bot to take expensive trades too, raise RISK_PER_TRADE_INR — but understand that also raises how much you can lose on one trade.

2. Monthly loss limit (MONTHLY_LOSS_LIMIT_INR, default Rs.15,000). The bot adds up your realized (closed) P&L for the current calendar month. Once losses for the month reach Rs.15,000, it stops offering new trades for the rest of the month — you'll see monthly loss limit hit. It resets on its own at the start of the next month.

- -> Any position already open is still managed normally (stop-loss, target, square-off all run). The limit only blocks *new* entries.
- -> During paper trading the limit counts paper P&L; once live, it counts live P&L.

These two are your hard guardrails. Treat a skip or a pause as the system doing its job — the whole point is to make a single bad trade, or a bad month, impossible to turn into a disaster.

Daily morning ritual

You'll get two Telegram messages from the system before market open:

8:30 AM IST — Token refresh nudge

A message like:

```
Good morning. Wed, 30 Apr 2026

Refresh Kite access token before 9:15 IST.

1. Tap to log in:
https://kite.zerodha.com/connect/login?api_key=XXX&v=3

2. SSH to Lightsail and run:
   cd ~/nifty-orb-alerts && source venv/bin/activate
   python auth.py
   nohup python orb_monitor.py > orb.log 2>&1 &
```

Steps to follow (~60 seconds)

1. Tap the Kite login link in the Telegram message → log in with Zerodha + 2FA. 2. After login, you land on a page that

may look broken — that's fine. Copy the full URL from the browser address bar (it contains request_token=...). 3. SSH to Lightsail from your laptop or AWS browser-based SSH:

```
ssh ubuntu@<lightsail-ip>
cd ~/nifty-orb-alerts
source venv/bin/activate
python auth.py
```

4. Paste the URL when prompted. You'll see Access token saved for today. 5. Start the monitor:

```
nohup python orb_monitor.py > orb.log 2>&1 &
```

6. Within a minute, you should get a Telegram alert: ORB monitor started (DRY_RUN) or (LIVE). If yes — you're done. Close the SSH window. The bot keeps running.

Why the bot keeps running after you close SSH

This is the part most people ask about, so here's the plain explanation:

- -> nohup (= "no hangup") tells Linux: "if my session ends, don't kill this process."
- -> The trailing & puts it in the background so the shell prompt comes back immediately.
- -> > orb.log 2>&1 sends both normal output and errors into the file orb.log so the process doesn't try to write to a terminal that's gone.

Net effect: you can close the SSH window, close your laptop lid, fly across the country — the bot keeps running on the Lightsail VM until either:

- -> You explicitly kill it (see "Daily shutdown" below), or
- -> The VM itself reboots (rare; AWS usually warns you).

If your VM does reboot, the bot will not restart automatically — see the optional systemd setup at the end of this handbook to fix that.

What the alerts mean

"ORB monitor started"

Confirmation the bot is alive. Includes mode (DRY_RUN/LIVE), futures symbol, lot size. No action.

"ORB formed" (around 9:30:30 AM)

```
ORB formed for NIFTY24MAYFUT
High: 22687.40
Low: 22641.25
Buffer: 10 pts | Re-arm: 15 pts
Watching 5-min closes for breakout/breakdown...
```

Your day's range is set. Wider ranges (>80 pts) often mean choppier days — be more selective.

"BREAKOUT (UP) — fire 1/2" — the real signal with a button

```
BREAKOUT (UP) -- fire 1/2
NIFTY24MAYFUT 5m close: 22693.10 > ORB High 22687.40 (+10 buf)
Vol: 142500 (avg 89200, x1.60)
VWAP: 22669.85
Spot Nifty: 22691.55
ITM-1 CE: NIFTY24MAY22650CE
Bid/Ask: 152.10/152.45
LIMIT: Rs.152.95 | Qty: 65 (risk <= Rs.5000)

[ BUY 65 CE @ Rs.152.95 ]   <- tap to place order
```

- -> fire 1/2 = first breakout in this direction today
- -> x1.60 = breakout candle volume was 1.6× the 20-candle avg — strong
- -> ITM-1 CE = 50 pts in the money (better delta than ATM for short holds)
- -> Bid/Ask = the live option quote at the moment the alert fired
- -> LIMIT Rs.152.95 = best_ask + 0.50 rounded to 0.05 tick — almost guaranteed to fill at the visible ask

Tap the button → bot places a BUY LIMIT order via Kite → confirmation message comes back:

```
Order placed: NIFTY24MAY22650CE BUY 65 @ LIMIT Rs.152.95
Order ID: 250502000123456
```

If the order fails (margin, rejected, network), you'll see:

```
ORDER FAILED for NIFTY24MAY22650CE: <error message from Kite>
```

…and you can place the trade manually in Kite.

"BREAKDOWN (DOWN) — fire 1/2"

Same shape but opposite — buys an ITM-1 PE (50 pts above spot).

"fire 2/2"

A re-entry breakout after price came back inside ORB by ≥15 pts. Often cleaner than fire 1 because the noise has been shaken out.

"Auto-order skipped" (any reason)

You'll see this if the safety guards rejected the auto-order:

- -> spread 7.2% > 5% — option spread too wide; place manually with a tight limit
- -> premium Rs.620 > MAX_PREMIUM Rs.500 — option too expensive (often deep ITM after a big move)
- -> quote fetch failed: ... — Kite API hiccup; refresh and place manually
- -> 1-lot risk Rs.5640 > risk cap Rs.5000 — one lot of this option would risk more than your per-trade cap (its premium is above ~Rs.256). This is by design — don't override it lightly (see "Position sizing" below)
- -> monthly loss limit hit (MTD Rs.-15200, limit Rs.15000) — the month's realized loss reached the cap; no new trades until next month

The alert still fires — you just don't get the button.

"Market closed. ORB monitor stopping."

End of day at 3:30 PM. Shows total fires.

Your trading checklist (still applies)

The button tap places the entry. Stops, exits, sizing — still your job:

1. Define stop-loss before the position fills. Standard rule: stop on the option = 30–40% of premium paid. On the futures level = re-entry into ORB range (use a Kite GTT for this). 2. Position size sanity. 1 lot of Nifty at premium Rs.150 = Rs.9,750 capital deployed. Stop at 30% = Rs.2,925 risk. That should be ≤ 1–2% of trading capital. If it's more, don't tap the button — the bot doesn't know your account size. 3. Set a target before the trade goes anywhere. Common: 1.5×–2× the risk. 4. Don't average down if it goes against you.

Daily shutdown (after market close)

The bot auto-sends Market closed. ORB monitor stopping. at 3:30 PM and the main loop exits. But the Python process may still be alive for a few seconds (the Telegram listener thread is a daemon and dies with the process — but only when the main thread exits cleanly).

Verify and clean up:

```
ssh ubuntu@<lightsail-ip>
ps aux | grep orb_monitor | grep -v grep
```

If you see a line, kill it:

```
pkill -f orb_monitor.py
```

Then verify it's gone:

```
ps aux | grep orb_monitor | grep -v grep
```

Empty output = clean.

Why this matters: if you start tomorrow's session without killing today's process, you'll have two bots running, doubled alerts, and possibly two BUY orders on the same button tap. Always kill before starting fresh.

Daily P&L review (V3)

Every paper / live trade is journaled to trades.db (a SQLite file in ~/nifty-orb-alerts/). One row per position with entry, exit, and P&L.

Quick review with report.py

```
cd ~/nifty-orb-alerts && source venv/bin/activate
python3 report.py                             # today
python3 report.py 2026-05-08                  # specific date
python3 report.py 2026-05-08 --mode PAPER     # filter by mode
```

Output is a per-trade table plus daily totals (wins / losses / win rate / net P&L).

Correcting a row (rare)

Back up first, then UPDATE. pnl is stored, not computed on read — always recompute it whenever you change entry_price or exit_price, or the row will be inconsistent:

```
cp trades.db trades.db.bak.$(date +%Y%m%d-%H%M)

sqlite3 trades.db "
BEGIN;
UPDATE trades
SET entry_price = <NEW_ENTRY>,
    pnl         = (exit_price - <NEW_ENTRY>) * qty
WHERE id = <ID>;
COMMIT;
"
```

Reconstructing a trade from orb.log

V3 writes structured, timestamped logs for every signal evaluation, fire, fill, position poll, and exit. After a trading day you can fully reconstruct any trade:

```
grep SIGNAL_FIRED orb.log        # all signals that fired today
grep "EXIT reason" orb.log       # all exits with realized P&L
grep position_tick orb.log       # minute-by-minute monitoring while holding
grep "tick candle" orb.log       # every 5-min signal evaluation (including skips)
```

Useful when a trade behaved unexpectedly — you'll see the exact ORB levels, volume ratio, VWAP, and signal decision for each candle.

Tuning the filters

All thresholds live in config.py on the Lightsail box. Edit, save, restart.

Setting	Default	What it does	When to change
VOLUME_MULTIPLIER	1.2	Breakout vol must be ≥ this × the 20-candle avg	Up to 1.5 for stricter; down to 1.0 if missing valid moves
LOOKBACK_CANDLES	20	Number of recent 5-min candles for the volume average	Rarely needs changing
MAX_FIRES_PER_DIRECTION	2	Max alerts per direction per day	1 for ultra-conservative; 3 if you want every clean re-break
BREAKOUT_BUFFER	10	Pts beyond ORB needed to count as a breakout	Higher = fewer fakeouts but later entries
REARM_BUFFER	15	Pts back inside ORB needed before re-arm	Higher = stricter re-entry filter
LOT_SIZE	65	Nifty lot size (NSE-defined)	Update if NSE changes the lot size
LIMIT_BUFFER	0.50	Rs. above best ask for the BUY limit	Higher = more fill certainty, slightly worse price

MAX_SPREAD_PCT	0.05	Skip auto-order if (ask−bid)/mid > this	Lower for stricter liquidity filter
MAX_PREMIUM	500	Skip auto-order if option premium > this	Catches expensive far-from-spot options
ORDER_PRODUCT	"MIS"	MIS = intraday auto-squareoff	Use "NRML" only if carrying overnight
DRY_RUN	True	Log instead of placing real orders	Flip to False after one clean DRY_RUN day
PAPER_TRADE	True	Simulate fills + auto-exits + journal to trades.db (no real orders)	Flip to False once 2 weeks of paper history look profitable

After editing config.py:

```
pkill -f orb_monitor.py
nohup python orb_monitor.py > orb.log 2>&1 &
```

Troubleshooting

"ORB monitor started" never arrives

```
tail -100 orb.log
```

Common errors:

Error contains	Fix
No valid access token for today	You forgot python auth.py. Run it.
TokenException	Token expired or wrong API key/secret. Re-run auth.py.
connection, timeout, network	Lightsail momentarily lost network. Restart.
No active Nifty futures found	Holiday or expiry edge case. Check NSE calendar.

Telegram alerts not arriving

```
python -c "from telegram_alert import send_alert; send_alert('test')"
```

-> Nothing on phone → bot token wrong, or you blocked the bot, or chat ID wrong.

-> Get Telegram send failed: ... → the error tells you.

Tapped the button but no order/confirmation came back

1. Check orb.log — look for Callback handler error: or ORDER FAILED. 2. Most likely cause: the access token expired mid-day (rare; tokens last ~24h). Re-run auth.py, restart the bot. The button you already tapped won't be retried — place that one manually. 3. Could also be that the bot process died. ps aux | grep orb_monitor | grep -v grep — if empty, restart.

"Auto-order skipped" on every alert

Spread or premium guards are too tight. Loosen MAX_SPREAD_PCT (e.g., to 0.07) or raise MAX_PREMIUM (e.g., to 800). Restart.

Process still running from yesterday

```
ps aux | grep orb_monitor | grep -v grep
pkill -f orb_monitor.py
```

Got two identical alerts within seconds

Two bots running. pkill -f orb_monitor.py (kills both), then start fresh.

Things you should NOT do

-> Don't flip DRY_RUN = False without one full day of dry-run validation. Real money, irreversible orders.
-> Don't run python auth.py after market open. The bot may already be exiting.
-> Don't push config.py to GitHub. It's in .gitignore. Secrets stay on Lightsail.
-> Don't paste your Kite API secret, access token, or TOTP into chats, screenshots, or anywhere outside config.py. If you do — rotate immediately at https://developers.kite.trade/apps.
-> Don't tap a button after the alert is more than ~30 seconds old — the underlying option price has moved and the LIMIT may now be too far below market to fill.
-> Don't let one losing day double your size the next day to "recover".

Risk discipline (read once a week)

-> Options decay daily. A 50% drop in the option premium is normal and frequent.
-> Win rate will not be 100%. Expect 40–55% winners. Profitability comes from average winner > average loser.
-> Daily loss cap: stop trading after 2 consecutive losses or 3% of capital, whichever comes first.
-> Weekly review: Sunday evening, review every alert vs. trades you took vs. P&L. Look for patterns (wide ORBs lose, tight ORBs win, etc.).

Optional: auto-restart with systemd

If you want the bot to survive a Lightsail VM reboot without you SSHing in to start it manually, create a systemd service. You still need to refresh the Kite token every morning — systemd just handles the process lifecycle.

One-time setup on the VM:

```
sudo tee /etc/systemd/system/orb-monitor.service > /dev/null <<'EOF'
[Unit]
Description=Nifty ORB Monitor
After=network-online.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/nifty-orb-alerts
ExecStart=/home/ubuntu/nifty-orb-alerts/venv/bin/python orb_monitor.py
Restart=on-failure
RestartSec=30
StandardOutput=append:/home/ubuntu/nifty-orb-alerts/orb.log
StandardError=append:/home/ubuntu/nifty-orb-alerts/orb.log

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable orb-monitor
```

Daily morning then becomes:

```
python auth.py                        # refresh token (still manual)
sudo systemctl restart orb-monitor    # picks up the new token
```

Daily shutdown:

```
sudo systemctl stop orb-monitor
```

Status check:

```
sudo systemctl status orb-monitor
```

If you're not comfortable with systemd, stick with nohup — it works fine for daily manual operation.

Roadmap

Version	Status	What it adds
V1	Done	Telegram alerts only
V1.5	Done	Re-arm logic — up to 2 fires per direction
V2	Done	5-min tracking; ITM-1 strike; 1-tap LIMIT order with safety guards; DRY_RUN flag

V3	Live now	PAPER_TRADE mode; auto SL/target/timeout exits; trades.db journal at alert-time price; structured orb.log; report.py daily summary
V4	Planned	Trailing SL; multi-symbol (BankNifty, FinNifty); holiday calendar
V5	Maybe	Web dashboard with live P&L and per-trade breakdown

Reference

- -> Repo: https://github.com/anilkumarjonnalagadda/nifty-orb-alerts (private)
- -> Kite developer console: https://developers.kite.trade/apps
- -> NSE holiday calendar: https://www.nseindia.com/resources/exchange-communication-holidays
- -> Bot logs on Lightsail: ~/nifty-orb-alerts/orb.log
- -> Cron logs (token reminder): ~/nifty-orb-alerts/cron.log